

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Context Engineering

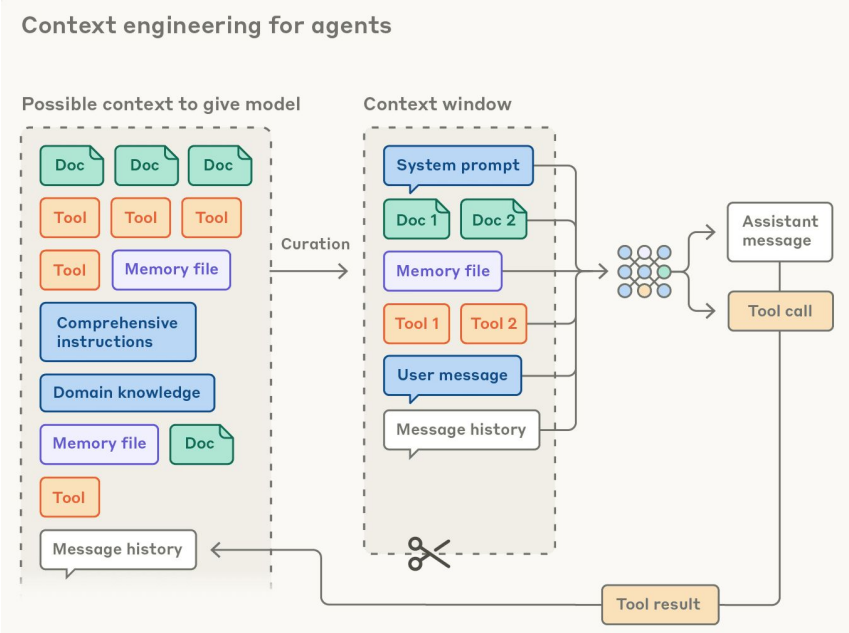
Outline

- Context engineering
- Context window
- Limits and failures
- Token economics

Context Engineering

- **Context:** set of tokens included when sampling from an LLM
- Curating what goes into the model's limited window at each step, from a dynamic pool of possible information
- Popularized in mid-2025 as the successor to "prompt engineering"
- Principle ([1]):
  - "Find the smallest set of high-signal tokens that maximizes the likelihood of the outcome you want"

Context Engineering



[1] Anthropic Engineering (2025). *Effective context engineering for AI agents.* https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

Context window and what goes in it

- The context window is the model's working memory for a single request
- Everything the model can use must fit inside it
- The window holds input and output together, including any hidden reasoning tokens
- API calls are stateless: each call resends everything the model should "remember"

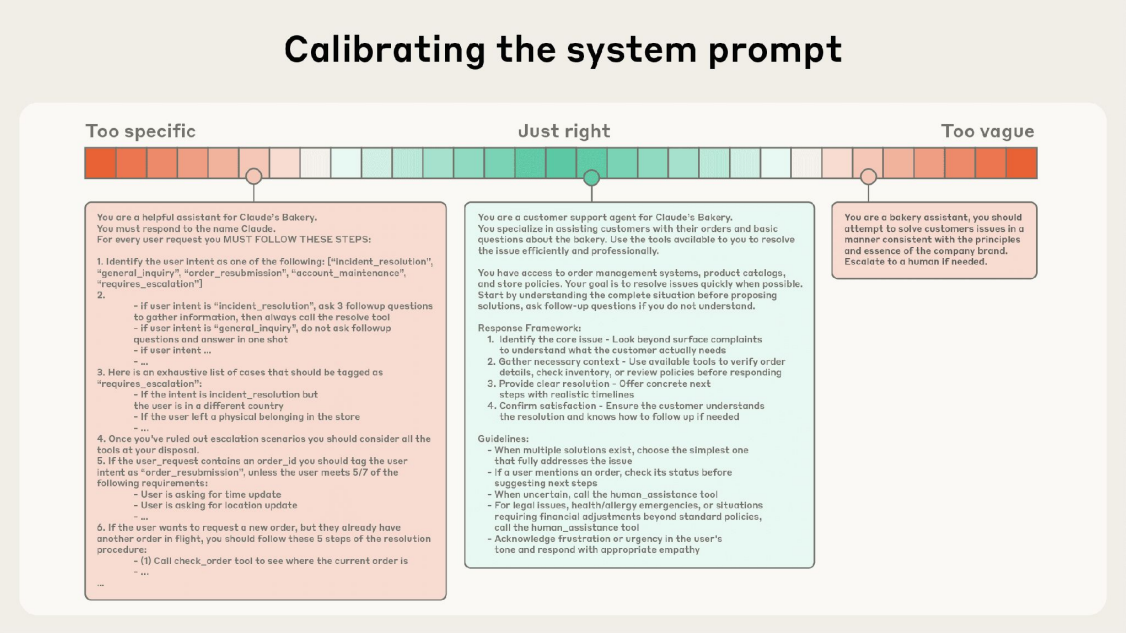
Context window: components

- System prompt
- Current user prompt
- Environment context
  - retrieved documents, files, tool definitions, tool results, memory, current date, conversation history
- Model output (including reasoning)

System Prompt

- The most stable part of the context; can be cached
- Sets the highest-level behavior for the model:
  - Role, goals, constraints, safety, output format, and rules for uncertainty
- Current practices:
  - Organize it into labeled sections (XML tags or Markdown headers) so the model can navigate it
  - Keep it lean. Anthropic reported removing a large share of Claude Code's system prompt for its newest models without measurable loss

System Prompt



[1] Anthropic Engineering (2025). *Effective context engineering for AI agents.* https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

User Prompt

- Often the *smallest* piece of the context but defines the task
- Best practices:
  - Be clear and direct
  - Include examples
  - Set the output format
  - Set the audience and a role/persona for the model
- Conversation history grows every turn. Old turns are rarely pruned even when no longer relevant

Environment Context

- Everything the system gathers *for* the model rather than what the user types:
  - Full-text papers, and search results
  - Tool outputs (API responses, code execution results)
  - Memory notes and files from earlier sessions
  - Metadata: current date, user profile, available resources
- Usually the largest and noisiest part of the window

Context window size

- Different models have different maximum context window sizes (measured in tokens)
  - 1 token ≈ 0.75 words

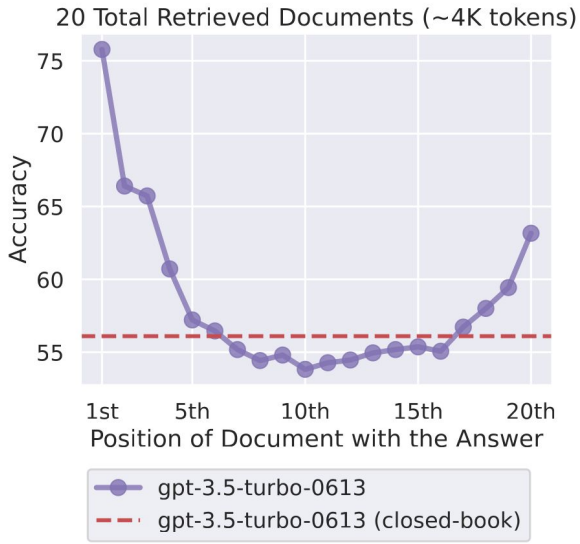
- Context window sizes for ALCF models:
  - https://docs.alcf.anl.gov/services/inference-endpoints/models/

Context window size: tradeoffs

- Larger context windows enable:
  - Processing longer documents
  - Maintaining longer conversation history
  - Including more retrieved information
- However:
  - More context increases cost/latency
  - Advertised window and effective window are different things
    - Effective capacity often degrades once prompts cross a certain threshold
  - A window being large doesn’t mean quality holds across it

The “lost in the middle” problem

- Models use information best when it sits at the **beginning or end** of the context
- Finding from 2023 study [3].
- Newer models have improved on this, but position effects have not disappeared



[3] Liu, N. F., et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts.*

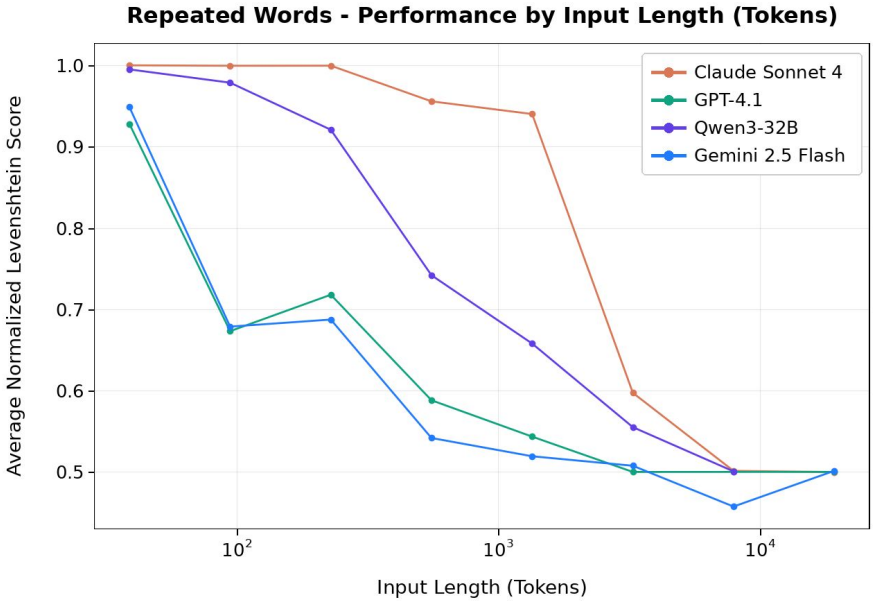
THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Lost in the middle: strategies

- **Strategic Reordering:** placing highest-priority instructions at the very beginning or the very end of the prompt
- **Chunk Summarization:** condense or summarize intermediate blocks of text
- **Cross-Encoder Reranking:** filter and rerank retrieved chunks before sending them to the LLM
- **Calibration Methods:** apply decoding or attention calibration frameworks to re-weight positional bias during retrieval and generation

Context rot

- As the number of tokens in the window increases, the model's ability to recall information decreases



[4] Hong, K., Troynikov, A., & Huber, J. (2025). *Context Rot: How Increasing Input Tokens Impacts LLM Performance.* Chroma Research.

Other context failures

- Other failure modes ([5]):
  - **Poisoning:** a hallucination or error enters the context and keeps being reused
  - **Distraction:** the model over-focuses on context, neglecting what it learned during training
  - **Confusion:** irrelevant content (such as too many tools) generates a low-quality response
  - **Clash:** new information conflicts with earlier information in the same context

Compression strategies

- Goal: keep the useful signal, drop the rest
  - **Tool-result clearing:** drops stale, re-fetchable data (such as old file reads) from inside the window
  - **Compaction:** summarizes a conversation nearing the context window limit and starts a new window with the summary
  - **Memory / notes:** moves information out of the window so it survives across sessions
  - **Prompt compression (e.g. LLMLingua):** remove low-information tokens

Summarization chains

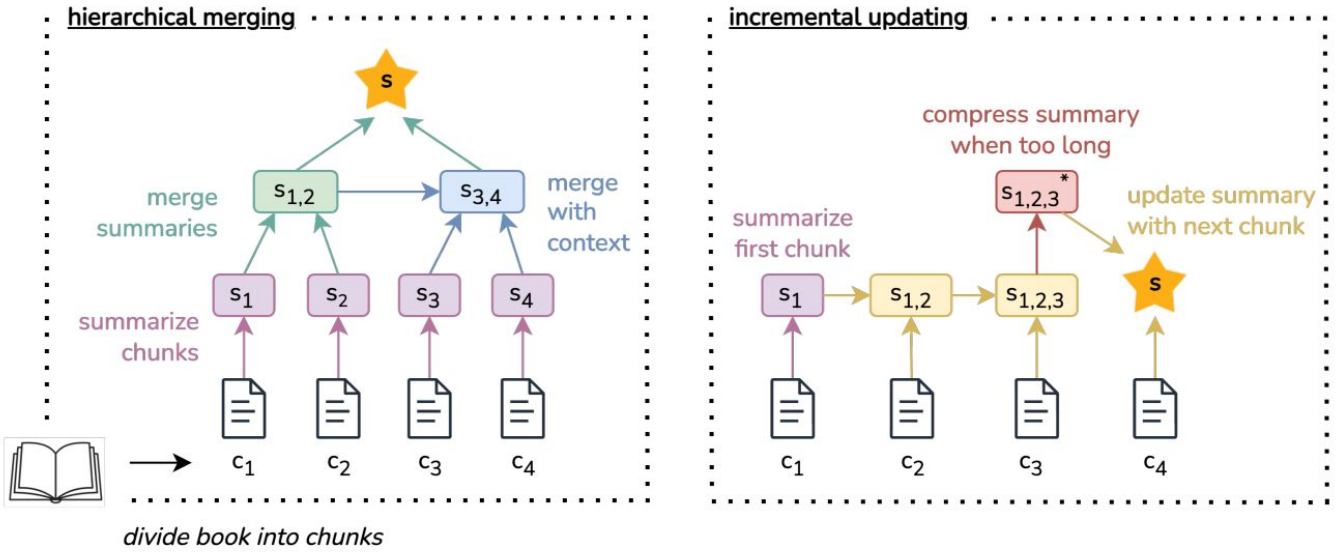
- Techniques for summarizing long texts, books, large collections of papers:
  - **Incremental updating:** at each step,

    new summary = summarize(old summary + new content)

    It keeps more detail but is less coherent
  - **Hierarchical merging:** summarize each chunk or paper independently, then summarize the summaries. More coherent but loses detail

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Summarization chains: example



[6] Chang, Y., et al. (2024). *BooookScore: A Systematic Exploration of Book-Length Summarization in the Era of LLMs.* ICLR. arXiv:2310.00785 https://arxiv.org/abs/2310.00785

THE UNIVERSITY OF CHICAGO
DATA SCIENCE INSTITUTE

Context engineering is cost engineering

- Most LLM APIs charge by tokens:
  - input tokens (prompt, system prompt, retrieved docs, tool outputs)
  - output tokens (the model's response)
- Output rates are usually several times more expensive than input rates
- Some models have separate rates for cache read/write
- Reasoning tokens are billed as output, even when not shown. A short visible answer can still consume many billable tokens
- Agent loops are the hidden multiplier: every turn resends the whole context, so cumulative input grows with the number of turns

Cost engineering: biggest levers

- **Prompt caching:** saves internal computation states for repeated *prefix* - such as system instructions, tool definitions, reference documents, or long conversation histories.
  - cache reads cost a fraction of normal input
  - automatic for OpenAI, explicit control for Anthropic/Google

Cost engineering: biggest levers

- **Model routing:** small model for high-volume steps, large model for hard cases
- **Trimming context:** every token you don’t send is saved on every call
- **Batch APIs:** cloud providers offer up to 50% off input and output cost for asynchronous work

Prices of current models

Provider	Model	Standard Input	Cached Input	Output
OpenAI	**GPT-6 Astra**	$10.00	$1.00	$50.00
	GPT-6.1 Sol	$2.00	$0.10	$10.00
Google	**Gemini 4 Argon**	$4.00	$0.20	$20.00
	Gemini Omni 1.2 Flash	$1.50	N/A	$9.00 (text) $12.00 (video)
Anthropic	**Claude Fable 5.2**	$8.00	$0.20	$40.00
	Claude Haiku 5.5	$0.10 (≤100K) $0.50 (>100K)	$0.01 (≤100K) $0.05 (>100K)	$0.50 (≤100K) $2.50 (>100K)
DeepSeek	**DeepSeek V4.1 Flash**	$0.30 (peak) $0.15 (off-peak)	$0.006 (peak) $0.003 (off-peak)	$1.20 (peak) $0.60 (off-peak)
Meta	**Muse Ember**	$0.10 (hosted)	$0.004 (hosted)	$0.50 (hosted)
Nex AGI	**Nex-N2.6**	$0.12 (hosted)	Varies by host	$0.40 (hosted)

Prices in USD per million tokens (MTok), as given in the slide breakdown.

How to estimate costs for a project

- **Formula, per call:**

```
cost = (uncached_in × P_in + cache_in × P_cache_in
        + output × P_out) / 1,000,000
```

- then apply modifiers (batch ×0.5, long-context rates)
- Claude Code and Pi provide tools for token usage and cost tracking

Activity 3: Context Rot Benchmarking

1. **Download recent papers from conference or journal**
   - Extract from: https://proceedings.mlsys.org/paper_files/paper/2026 (you can use prompt)

2. **Build Evaluation Pipeline**
   - Extract free-text questions/answers/paper source info to a .json file
   - Spot check questions
   - Set up a LLM-as-a-judge pipeline
   - Open coding agent in paper folder without questions/answers

3. **Explore coding agent tools for retrieval**
   - Open coding agent in paper folder without access to answers

References

[1] Anthropic Engineering (2025). *Effective context engineering for AI agents.* https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

[2] Harrison Chase (LangChain, 2025). *The rise of "context engineering".* https://www.langchain.com/blog/the-rise-of-context-engineering#what-is-context-engineering

[3] Liu, N. F., et al. (2024). *Lost in the Middle: How Language Models Use Long Contexts.* TACL. arXiv:2307.03172 https://arxiv.org/abs/2307.03172

[4] Hong, K., Troynikov, A., & Huber, J. (2025). *Context Rot: How Increasing Input Tokens Impacts LLM Performance.* Chroma Research. https://www.trychroma.com/research/context-rot

[5] Breunig, D. (2025). *How Long Contexts Fail* (and *How to Fix Your Context*). dbreunig.com. https://www.dbreunig.com/2025/06/22/how-contexts-fail-and-how-to-fix-them.html

[6] Chang, Y., et al. (2024). *BooookScore: A Systematic Exploration of Book-Length Summarization in the Era of LLMs.* ICLR. arXiv:2310.00785 https://arxiv.org/abs/2310.00785